Задание 2-3 (Рабочая тетрадь 5)

Задание 3 (Рабочая тетрадь 5)

Задача: Разработайте программу на интерпретируемом языке программирования (например, Python, JavaScript или Ruby), которая будет выполнять простую задачу, такую как обработка текстового файла или вывод данных в графическом интерфейсе.
Программа должна быть написана таким образом, чтобы она могла работать на любой платформе, где установлен интерпретатор соответствующего языка.

Содержание

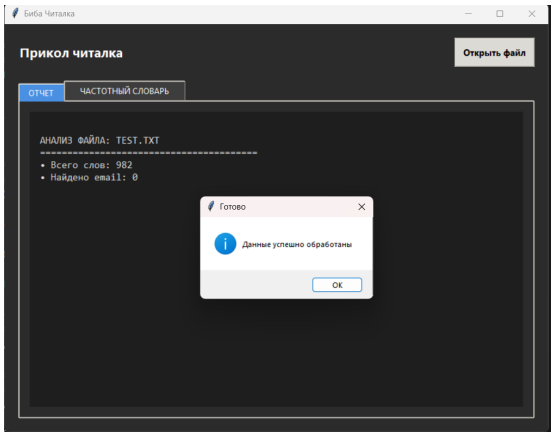
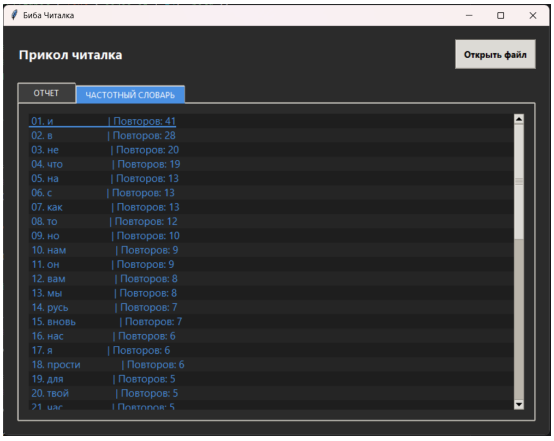
1. Принцип работы
2. Листинг кода
3. Пример использования

Реализация

Для выполнения данного задания было разработано кроссплатформенное приложение на языке Python с графическим интерфейсом на `tkinter`.
Программа **«Прикол читалка»** (Modern Text Analyzer) позволяет: - Открыть текстовый файл в формате `.txt` - Выполнить статистический анализ содержимого: - Подсчёт общего количества слов - Поиск email-адресов - Выделение **50 наиболее часто встречающихся слов** с частотой повторов - Вывести результаты в двух вкладках: - **ОТЧЁТ** – текстовый лог с общей информацией - **ЧАСТОТНЫЙ СЛОВАРЬ** – список топ-50 слов в стилизованном виде

Приложение использует только **стандартные библиотеки Python**: `tkinter`, `re`, `threading`, `collections`. Это обеспечивает полную переносимость на любую платформу (Windows, Linux, macOS), где установлен Python 3.6+.

Скриншоты работы

Главное окно (вкладка «ОТЧЕТ»)	Вкладка «ЧАСТОТНЫЙ СЛОВАРЬ»
	

Принцип работы

Приложение построено на базе класса `ModernAnalyzer` и работает следующим образом:

1. **Инициализация интерфейса**
2. Создаётся главное окно с тёмной темой, настраиваются стили `ttk`.
3. Формируются две вкладки: лог-консоль и список частотных слов.

4. Внизу окна располагается прогресс-бар.

5. **Выбор файла**

6. По нажатию кнопки **«Открыть файл»** вызывается диалоговое окно выбора `.txt` файла.

7. **Асинхронная обработка**

8. Чтение и анализ выполняются в отдельном потоке (`threading.Thread`), чтобы интерфейс не «зависал».

9. Прогресс-бар обновляется через `root.after()`, что безопасно для многопоточности.

10. **Анализ текста**

11. Извлечение всех слов: `re.findall(r'\w+', data.lower())`
12. Поиск email'ов: `re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', data)`
13. Подсчёт частоты слов с помощью `collections.Counter`.

14. Выбор топ-50 наиболее частотных слов.

15. **Вывод результатов**

16. В первую вкладку построчно записываются: путь к файлу, количество слов, количество email'ов и пример первого email.

17. Во второй вкладке формируется нумерованный список слов с количеством повторений. Чётные строки подсвечиваются более тёмным цветом для удобства чтения.

18. **Завершение**

19. По окончании анализа прогресс-бар заполняется на 100% и появляется всплывающее сообщение «Готово».
20. Любая ошибка (например, отсутствие файла, проблемы с кодировкой) перехватывается и выводится в `messagebox`.

Листинг кода

Исходный код основного файла: main.py

```
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
import re
import threading
from collections import Counter

class ModernAnalyzer:
    # ... (полный код приведён в приложении)
```

Пример использования

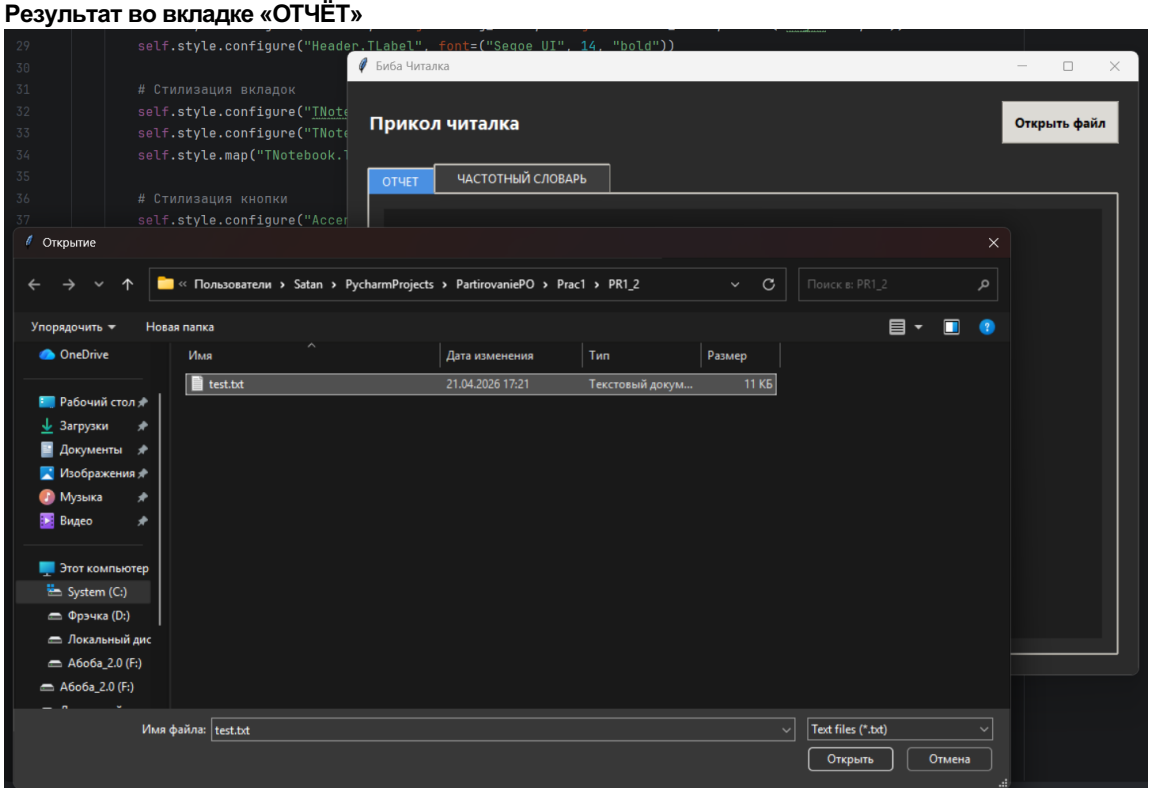
1. Запустите main.py.
2. Нажмите кнопку **«Открыть файл»**.
3. Выберите любой текстовый файл (например, test.txt с произвольным содержимым).
4. Дождитесь завершения анализа — прогресс-бар заполнится.
5. Переключайте вкладки для просмотра отчёта и частотного словаря.

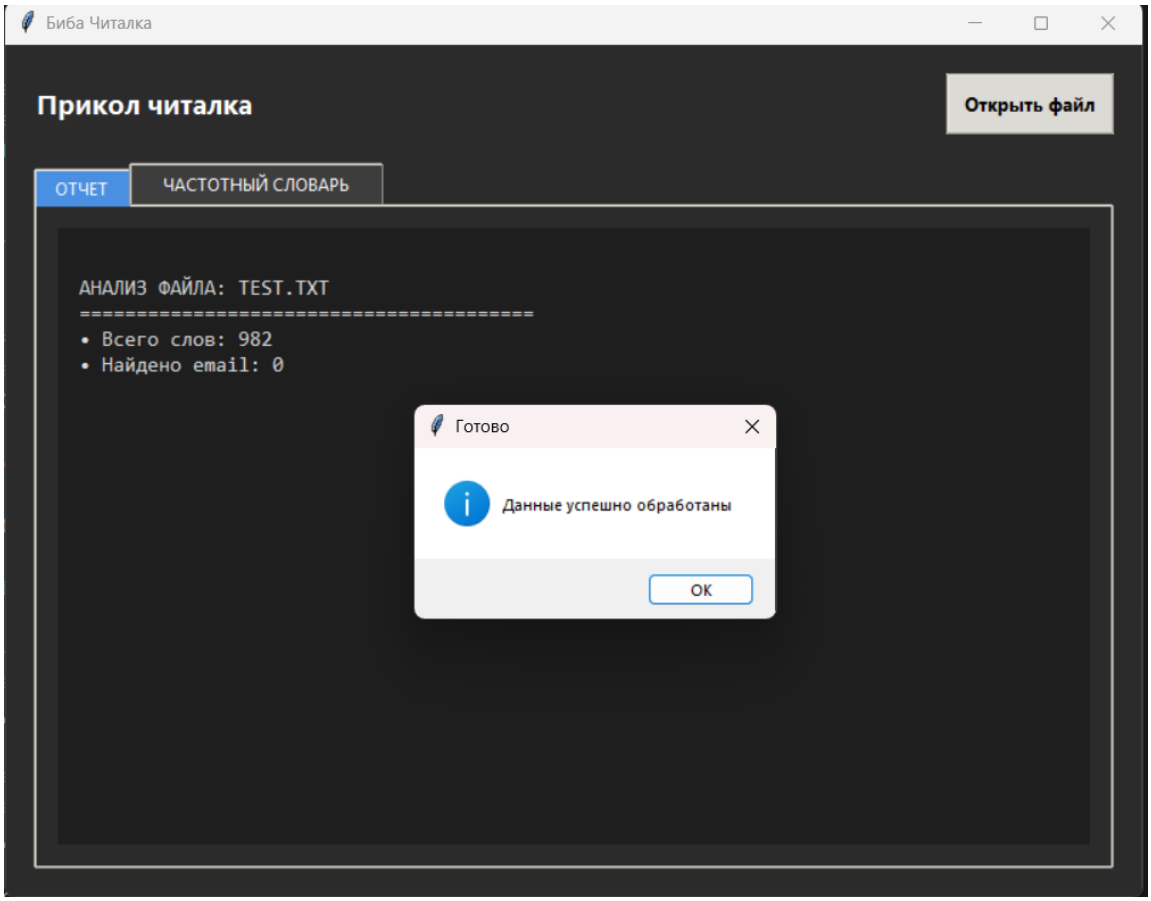
Пример содержимого test.txt:

```
На европейские события в 1854 году

 С чего взялась всесветная беда?
 Кто виноват, кто первый начинает?
 Народ вы умный, всякой это знает,
 Да славушка пошла об вас худа!
 Уж лучше бы в покое дома жить
 Да справиться с домашними делами!
 Ведь, кажется, нам нечего делить
 И места много всем под небесами.
 К тому ж и то, коль всё уж поминать:
 10 Смешно французом русского пугать!

 Знакома Русь со всякою бедой!
 Случалось ей, что не бывало с вами.
 Давил ее татарин под пятой,
 А очутился он же под ногами.
 Но далеко она с тех пор ушла!
 Не в мерку ей стать вровень даже с вами;
 Заморский рост она переросла,
 Тянуться ль вам в одно с богатырями!
```

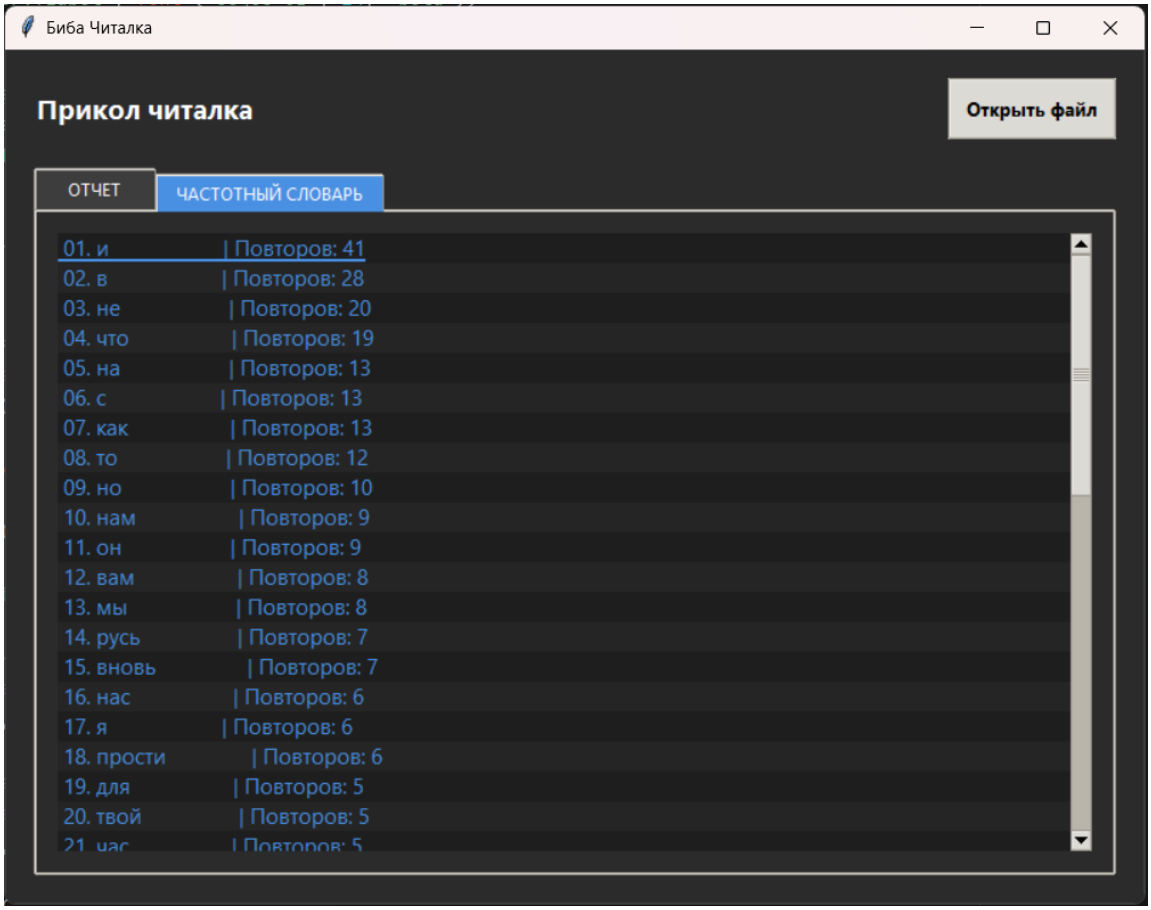
```
Попробуйте на нас теперь взглянуть,
20 Коль не боитесь голову свихнуть!
```

Результат во вкладке «ОТЧЁТ»





Результат во вкладке «ЧАСТОТНЫЙ СЛОВАРЬ»



Задание 2 (Рабочая тетрадь 5)

Задача: Разработайте программу, которая будет использовать как переиспользование бинарных файлов, так и переиспользование исходного кода.

Программа должна состоять из двух частей: основной программы (написанной на C или C++) и библиотеки, которая будет использоваться в основной программе.

Библиотека должна быть собрана отдельно и использоваться как бинарный файл на разных платформах.

Содержание

1. Структура проекта
2. Сборка и использование
3. Сборка статической библиотеки
4. Сборка динамической библиотеки
5. Сборка основной программы
6. Принцип работы
7. Листинг кода
8. Пример работы

Реализация

Для выполнения задания разработана программа на языке **C++**, состоящая из:

- **Библиотеки `mylib`**, предоставляющей две функции:
- `int add(int a, int b)` – сложение двух целых чисел.
- `void print_message(const char* msg)` – вывод сообщения с префиксом `"Library says: "`.
- **Основной программы `main.cpp`**, которая подключает библиотеку, вызывает её функции и выводит результаты.

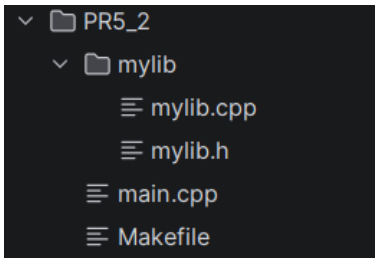
Библиотека собрана таким образом, что её можно использовать в двух режимах: 1. **Переиспользование исходного кода** – подключение заголовочного файла `mylib.h` и компиляция вместе с `mylib.cpp`. 2. **Переиспользование бинарных файлов** – предварительная сборка библиотеки в статический (`.a` / `.lib`)

или динамический (`.so` / `.dll`) файл, после чего линковка с основной программой без перекомпиляции библиотеки.

Кроссплатформенность обеспечивается макросами `_WIN32` в заголовочном файле (для экспорта/импорта символов в Windows) и универсальным `Makefile`, поддерживающим **Linux**, **macOS** и **Windows (MSYS2/MinGW)**.

Структура проекта

```
project/
├── mylib/
│   ├── mylib.h        # Заголовочный файл библиотеки (с макросами экспорта)
│   └── mylib.cpp      # Реализация библиотеки
├── main.cpp           # Основная программа, использующая библиотеку
└── Makefile           # Универсальный Makefile для сборки под разные ОС
```



Сборка и использование

Требования

- Компилятор с поддержкой C++11: `g++`, `clang++` или `MinGW` (для Windows).
- Утилита `make` (в Windows можно использовать `mingw32-make`).

Сборка статической библиотеки

Linux / macOS:

```
make static          # создаёт lib/mylib.a
```

Windows (MinGW):

```
mingw32-make static  # создаёт lib/mylib.a
```

Сборка динамической библиотеки

Linux / macOS:

```
make shared          # создаёт lib/mylib.so (или .dylib на macOS)
```

Windows (MinGW):

```
mingw32-make shared  # создаёт bin/mylib.dll и lib/mylib.dll.a (import library)
```

Сборка основной программы

Программа может линковаться **статически** (библиотека встраивается в исполняемый файл) или **динамически** (библиотека загружается во время выполнения).

Статическая линковка:

```
make main-static     # создаёт main_static
```

Динамическая линковка:

```
make main-shared     # создаёт main_shared
```

Во втором случае при запуске `main_shared` исполняемый файл должен найти библиотеку: - На Linux: `export LD_LIBRARY_PATH=./lib:$LD_LIBRARY_PATH && ./main_shared` - На Windows: скопировать `mylib.dll` в ту же папку, что и `main_shared.exe`.

Универсальная цель `all`

```
make all             # собирает статическую библиотеку и статически слинкованную программу
make clean           # удаляет все сгенерированные файлы
```

Принцип работы

1. Заголовочный файл [mylib.h]

Использует условную компиляцию для определения макроса `MYLIB_API`:

```
#ifdef _WIN32
  #ifdef MYLIB_EXPORTS
    #define MYLIB_API __declspec(dllexport)
  #else
    #define MYLIB_API __declspec(dllimport)
  #endif
#else
  #define MYLIB_API
#endif
```

- При сборке самой библиотеки (`MYLIB_EXPORTS` определён) символы **экспортируются**.
- При использовании библиотеки из программы (`MYLIB_EXPORTS` не определён) символы **импортируются**.
- На платформах, отличных от Windows, макрос пустой (экспорт/импорт не требуется).

2. Реализация [mylib.cpp]

Содержит простые функции:

```
int add(int a, int b) { return a + b; }
void print_message(const char* msg) { std::cout << "Library says: " << msg << std::endl; }
```

3. Основная программа [main.cpp]

Подключает библиотеку через `#include "mylib/mylib.h"` и вызывает её функции:

```
int result = add(3, 4);
std::cout << "Result: " << result << std::endl;
print_message("Hello from main!");
```

4. Makefile

```
CXX = g++
CXXFLAGS = -Wall -std=c++11 -I.

# Динамическая библиотека
shared: mylib/mylib.cpp
    $(CXX) -fPIC -shared -DMYLIB_EXPORTS $< -o libmylib.so

# Статическая библиотека
static: mylib/mylib.cpp
    $(CXX) -c $< -o mylib.o
    ar rcs libmylib.a mylib.o

# Статическая линковка программы
main-static: main.cpp static
    $(CXX) $< libmylib.a -o main_static

# Динамическая линковка программы
main-shared: main.cpp shared
    $(CXX) $< -L. -lmylib -o main_shared

clean:
    rm -f *.o *.a *.so main_*
```

Полный Makefile с поддержкой Windows (через условные конструкции) прилагается в проекте.

Листинг кода

`mylib.h` – заголовочный файл библиотеки

```
#ifndef MYLIB_H
#define MYLIB_H

#ifdef _WIN32
  #ifdef MYLIB_EXPORTS
    #define MYLIB_API __declspec(dllexport)
  #else
    #define MYLIB_API __declspec(dllimport)
  #endif
#else
  #define MYLIB_API
#endif

MYLIB_API int add(int a, int b);
MYLIB_API void print_message(const char* msg);

#endif
```

`mylib.cpp` – реализация библиотеки

```
#include <iostream>
#include "mylib.h"

int add(int a, int b) {
    return a + b;
}

void print_message(const char* msg) {
    std::cout << "Library says: " << msg << std::endl;
}
```

`main.cpp` – основная программа

```
#include <iostream>
#include "mylib/mylib.h"

int main() {
    int result = add(3, 4);
    std::cout << "Result: " << result << std::endl;

    print_message("Hello from main!");

    return 0;
}
```

Пример работы

Команды для Linux:

```
make static               # сборка статической библиотеки
make main-static          # сборка программы со статической линковкой
./main_static
```

Вывод в консоли:

```
Result: 7
Library says: Hello from main!
```

Команды для Windows (MinGW):

```
mingw32-make shared       # сборка mylib.dll
mingw32-make main-shared  # сборка main_shared.exe
# Скопировать mylib.dll в ту же директорию, где находится main_shared.exe
main_shared.exe
```

Переиспользование бинарных файлов

Если библиотека уже собрана (`libmylib.a` или `libmylib.so` / `mylib.dll`), её можно использовать **без повторной компиляции**:

1. Скопировать заголовочный файл `mylib.h` и бинарный файл библиотеки в другой проект.
2. Подключить заголовок и указать путь к библиотеке при линковке.

Пример использования статической библиотеки в новом проекте:

```
g++ new_main.cpp -I/path/to/include -L/path/to/lib -lmylib -o new_program
```

Пример использования динамической библиотеки (Linux):

```
g++ new_main.cpp -I/path/to/include -L/path/to/lib -lmylib -Wl,-rpath,/path/to/lib -o new_program
```

На Windows достаточно поместить `mylib.dll` в папку с исполняемым файлом или в системный PATH.

Таким образом, достигается **переиспользование бинарного кода** без необходимости распространять исходники библиотеки.